*Colegio Salesiano "San Ignacio"*
*Avda. María Auxiliadora nº7*
*CP 11009, Cádiz*

salesianos
COLEGIO SAN IGNACIO
CÁDIZ

***Sección de Formación Profesional***

Nombre: _____Fecha: 17 de Noviembre de 2023

Curso y ciclo: 1º Desarrollo de Aplicaciones Multiplataforma (DAM) Nota:

Módulo: Programación

**Prueba 3 RA3**

**PE 3. Desarrollar programas que hagan uso de estructuras de selección, estructuras de repetición, control de excepciones y recursividad. Duración: 2h 30m**

**Ejercicio 1. (1,6 puntos)**

**Desarrolla una clase llamada Ejercicio1, que tenga un método llamado prueba con el siguiente código.**

```
public static void prueba(String num) {
        String x = null;
        System.out.println(x.charAt(0));
        float decimal=Float.parseFloat(num);
        prueba("4");
        ArrayList<byte[]> lista = new ArrayList<>();
       while (true) {
           lista.add(new byte[1048576]);
       }
```

**Desde el método main, llama al método prueba, por ejemplo con prueba("Hola"). Detecta que excepciones se provocan en el código correspondiente al método prueba y gestiónalas desde dicho método. Se gestionará cada excepción con un mensaje que indique la línea del código donde se provoca la excepción y un mensaje que personalicéis vosotros (Por ejemplo, si la excepción fuese OneException por una división por cero, podríamos enviar por consola el mensaje 12. OneException → No podemos dividir por cero).**

**Ejercicio 2. (2 puntos)**

**La secuencia de Collatz, también conocida como conjetura de Collatz o problema 3n + 1, es una secuencia matemática que comienza con un número entero positivo n y sigue las siguientes reglas:**

**Por ejemplo, si comenzamos con n = 6, la secuencia se vería de la siguiente manera:**

**6 (par) => Dividido por 2 => 3**
**3 (impar) => Multiplicado por 3 y sumar 1 => 10**
**10 (par) => Dividido por 2 => 5**
**5 (impar) => Multiplicado por 3 y sumar 1 => 16**
**16 (par) => Dividido por 2 => 8**
**8 (par) => Dividido por 2 => 4**
**4 (par) => Dividido por 2 => 2**
**2 (par) => Dividido por 2 => 1**

**Hay valores de "n" que no paran nunca el programa, pero nosotros no vamos a tener en cuenta esos valores y vamos a parar cuando lleguemos al número 1 (si introduces por teclado algún número que no llegue al 1 y no para, no lo tengas en cuenta) Por ejemplo, para n=6:**
**El resultado de nuestro programa sería: 6 3 10 5 16 8 4 2 1. Desarrolla un programa que calcule la secuencia de Collatz para un "n" introducido por teclado, mediante RECURSIVIDAD.**

**<u>Ejercicio 3.</u> (1,5 puntos)**

**Una compañía dedicada al alquiler de automoviles cobra un precio fijo de $300.000 para los primeros 300 km de recorrido. Para más de 300 km y hasta 1000 km, cobra un precio adicional de $15.000 por cada kilómetro que excede de 300. Para más de 1000 km cobra un precio adicional de $10.000 por cada kilómetro que excede de 1000. Los precios no incluyen el 20% del impuesto general a las ventas, IVA. Desarrolla un programa que reciba el número de kilómetros recorridos (puede contener números decimales) y muestre por pantalla el precio total a pagar con IVA, con solo dos decimales.**

**<u>Ejercicio 4.</u> (2,5 puntos)**

**Desarrolla un programa que pida por teclado un número entero positivo que estará en representación binaria y lo convierta a decimal. Tenemos que controlar que ese número sea un número entero y positivo, y en caso de no cumplir alguna de las dos condiciones anteriores, volvemos a pedir el número para realizar su conversión.**

**Por ejemplo, si introduzco el número 100, el programa me devolverá 4**

**¿Cómo se calcula? Con potencias de 2 (100 → $2^2*1+2^1*0+2^0*0=4+0+0=4$)**

**PISTA: A lo mejor es interesante crear un método para obtener cuantos dígitos tiene el número binario.**

**<u>Ejercicio 5.</u> (2,4 puntos)**

**Diseña una aplicación que dibuje el triángulo de Pascal, para 'n' filas. Numerando las filas y elementos desde 0, la fórmula para obtener el m-ésimo elemento de la n-ésima fila es:**
**E(n,m)= n! / m!(n-m)!, sabiendo que el factorial de un número, por ejemplo 5, se calcula de la siguiente forma:**
**5!=5*4*3*2*1;**

**1**
**1 1**
**1 2 1**
**1 3 3 1**
**1 4 6 4 1**

**<u>Corrección</u>**
**Ej1 (Detectar excepciones) 0,4p**
**Ej1 (Gestionar cada excepción en el método) 0,8p**
**Ej1 (Mensaje correcto para cada excepción) 0,4p**
**Ej2 (Planteamiento recursividad, caso base) 0,5p**
**Ej2 (Planteamiento recursividad, caso genérico) 1p**
**Ej2 (Imprimir secuencia correcta) 0,5p**
**Ej3 (Planteamiento estructuras correctas) (0,75p)**
**Ej3 (Cálculo resultado correcto) (0,5p)**
**Ej3 (Imprimir por pantalla resultado) (0,25p)**
**Ej4 (Control número entero positivo) (0,5p)**
**Ej4 (Obtener cifras número) (0,5p)**
**Ej4 (Algoritmo conversión a decimal) (1,5p)**
**Ej5 (Calcular factorial de un número) (0,2p)**
**Ej5 (Obtener el término correspondiente con n y m) (1p)**
**Ej5 (Pintar el triángulo tal como se pide) (1,2p)**